

Università degli Studi Milano-Bicocca

Dipartimento di Fisica “Giuseppe Occhialini”

Corso di Laurea Triennale in Fisica



Qudit-Based Quantum-Classical Neural Networks for Classification

Relatore:

Prof. Andrea Giachero



Correlatore:

Dott. Rodolfo Carobene

Candidato:

Lorenzo Sabadini

Matricola: 898040

Anno Accademico 2025/2026

Summary

Quantum Machine Learning (QML) merges a branch of artificial intelligence with quantum computing, one of the most prominent fields of recent decades. This thesis work aims to investigate a new approach for the implementation of quantum-classical learning models: the use of quantum circuits based on *qudits*, which are a multi-level extension of *qubits*.

The term *Machine Learning* was coined in the mid-twentieth century to describe the emerging concept of "machines that learn from a dataset". Since then, research has made great strides and today machine learning algorithms are widely used in both research and commercial fields. These models are limited by the difficulty of managing large datasets and very long training times, but QML is a promising path to overcome those limits. Quantum computers harness the principles of quantum mechanics, such as entanglement, superposition of states and wave function interference, leading to computational advantages which overcome classical computers in certain classes of problems. While quantum machine learning implementations are many, this study focuses on a Neural Network model for image classification. A neural network architecture consists of a sequence of layers; each layer is formed by a specific number of neurons that progressively transform the input data to produce the final output. A Hybrid Quantum-Classical Neural Network can be obtained by replacing one or more layers with a quantum circuit.

Backpropagation algorithm is a fundamental part of the learning process of a neural network; it calculates and combines, in reverse, the gradients of all the transformations applied by the layers, to then update the decision parameters of the network. Within a hybrid neural network, the gradient definition of a quantum operation is provided by the *parameter shift-rule*, so that the parameters of the quantum layer can also be processed and updated by backpropagation.

The fundamental unit of quantum computer is the qubit, which is a two-level physical state described by elements of a two-dimensional Hilbert space (H^2). Qudits, on the other hand, are physical states with an arbitrary number of energy levels defined in a d -dimensional Hilbert space (H^d). Usually, real physical systems are not binary and so are better described by qudits; besides, the amount of information they can store is higher, reducing the number of qudits required in a circuit. Indeed, qudits can also be used in computation, provided that the quantum logic gates are redefined in the new higher-dimensional space (H^d). Replacing qubits with qudits in a circuit inevitably leads to fewer gates between neighboring qudits (inter-qudit gates), and an increase in single-qudit gates. Whether this exchange is advantageous or disadvantageous depends on the type of gates used, but typically inter-qudit gates introduce greater error into circuits compared to single-qudit gates, so reducing their number would decrease the impact of environmental noise.

The circuits implemented, based on a single qudit, are reported below:

- Generalized Hadamard gate followed by rotations between pairs of energy levels around the X axis applied to a three dimensional-qudit and a five-dimensional qudit.
- Generalized Hadamard gate followed by combinations of rotations between pairs of energy levels around the X, Y, and Z axes applied to a three-dimensional qudit.
- Generalized Hadamard gate followed by a universal gate for qudits applied to a three-dimensional qudit and a five-dimensional qudit.

Tests carried out in this work aim to compare: total training time of the network, accuracy achieved, and reliability, defined as the number of successful training runs out of a given number of attempts. The results indicate that varying the number of parameters does not lead to a corresponding loss of accuracy; however, the training time increases considerably. Increasing the system dimensionality led to a worsening of reliability; nevertheless, it was possible to classify up to five images using a single qudit without a substantial drop in accuracy.

Even though these results are promising, further analysis is required to confirm whether the observed behaviors remain stable, regardless of factors other than the number of parameters and system size.

Contents

1	Classical and Quantum Machine Learning	1
1.1	Machine learning	1
1.1.1	Fundamentals of Artificial Neural Networks	2
1.1.2	Neural Networks Learning Process	5
1.1.3	Pytorch library	7
1.1.4	Image Recognition and the MNIST Dataset	8
1.2	Quantum Machine Learning	9
1.2.1	Introduction to Quantum Computing	9
1.2.2	Qudits and High-Dimensional Hilbert Spaces	12
1.2.3	Cirq library	13
1.2.4	Hybrid Neural Networks	14
1.2.5	Backpropagation with quantum layers	16
2	Qudit-Based Neural Networks	17
2.1	Introduction to qudits	17
2.1.1	Universality condition	18
2.1.2	Generalized Gates	18
2.2	Qudit Neural Network implementation	22
2.2.1	Initial State Preparation	23
2.2.2	Qudit and Custom Gate Implementation in Cirq	24
2.2.3	Loss function	27
2.3	Tests and results	28
2.3.1	Quantum layer with [entangled_qubit] circuit	29
2.3.2	Quantum layer with [H_X_ij] circuit	29
2.3.3	Quantum layer with [H_X_ij_d=5] circuit	30
2.3.4	Quantum layer with [H_R_ij] circuit	31
2.3.5	Quantum layer with [H_R_ij_dim=5] circuit	31
2.3.6	Quantum layer with [H_XYZ_ij] circuit	32

3	Discussion and Outlook	34
3.1	Discussion of Experimental Results	34
3.1.1	The role of Generalized Hadamard Gate	38
3.1.2	Future Work	38
3.2	Conclusions	40

Chapter 1

Classical and Quantum Machine Learning

1.1 Machine learning

Machine Learning (ML) is a subfield of Artificial Intelligence (AI) concerned with the development and study of statistical algorithms that learn from datasets and generalize to unseen data by identifying patterns rather than relying on explicit rules. Historically, the discipline originated from efforts to model human cognition and biological learning processes. A foundational contribution occurred in 1949, when psychologist Donald Hebb published *The Organization of Behavior*, proposing a theoretical structure based on interactions among nerve cells. Hebbian theory established that learning relies on neuronal connections and synaptic plasticity, defined as the ability of synapses to strengthen or weaken over time. Inspired by these biological mechanisms, researchers developed various machine learning algorithms. Among these, Artificial Neural Networks (ANNs) represent the model most structurally analogous to biological neural networks, forming the foundation of this thesis.

Machine learning algorithms fall into five main categories:

- *Supervised learning*: the model learns through a set of labeled data, called *training data*, where the labels represent a characteristic of the data that we already know and that we want the model to predict. In this way, the model's prediction can be checked by the model itself, which is able to modify its parameters and get a better answer, learning the connection between the data and the labels.
- *Unsupervised learning*: in this case the model is able to train with unlabeled data. The algorithm is set to look for some interesting structure and it tries to determine correlations between the data without any supervision.

- *Self-supervised learning*: also called predictive learning algorithms, they receive unlabeled data but they generate labels automatically, transforming unsupervised problems into supervised ones.
- *Reinforcement learning*: a type of dynamic programming that trains algorithms using a system of reward and punishment. An agent takes actions in a specific environment, and it is rewarded or penalized for its actions based on an established metric, encouraging the agent to continue good practice and avoid bad ones.
- *Semi-supervised learning*: is a combination of supervised and unsupervised learning. These algorithms are trained on a small labeled dataset and a large unlabeled dataset. The model uses unsupervised learning to identify data clusters and then uses supervised learning to label them.

In this thesis, we work in a Supervised Learning context. Specifically, we implement a *Neural Network* (NN) designed to classify handwritten digits. Therefore, the following section delves deeper into analyzing this kind of Machine Learning structure.

1.1.1 Fundamentals of Artificial Neural Networks

Artificial Neural Networks are the core of what is called *Deep Learning* [1]. This section and the following one discuss their structure and the way they learn, providing an overall theoretical overview and the mathematical tools necessary to understand their functioning.

A Neural Network is a combination of connected neurons. Each neuron and each connection between them have associated parameters called *bias* (b) and *weights* (w). These parameters translate the mechanism of synaptic plasticity into numerical functions; in fact, the algorithm learns by changing them through the training process. Neurons are organized in layers, as shown in Figure 1.1, in this way: the first layer receives the input data, which is why it is called *input layer*; in the middle there are the *hidden layers*, so named because they do not directly interface with the user, and in the end the *output layer* generates the final output. Since no analytical rule dictates the optimal number of layers and neurons, determining the best architecture typically relies on empirical experimentation and hyperparameter tuning.

A closer inspection of the neural network architecture illustrated in Figure 1.1 reveals several defining structural characteristics worth exploring in detail:

- each neuron and each connection has its own distinct bias or weight;
- the vector $\bar{x} = x_1, x_2, x_3, x_4$ represents the input data, while y_1 and y_2 denote the outputs of the whole network;

- each neuron in the input layer receives a single element of the input vector without performing any computation, serving solely to pass the features forward to the subsequent layer;
- $b_i^{(l)}$ represents the bias of neuron i in layer l ;
- $w_{jk}^{(l)}$ represents the weight connecting neuron j in layer $l - 1$ to neuron k in layer l .

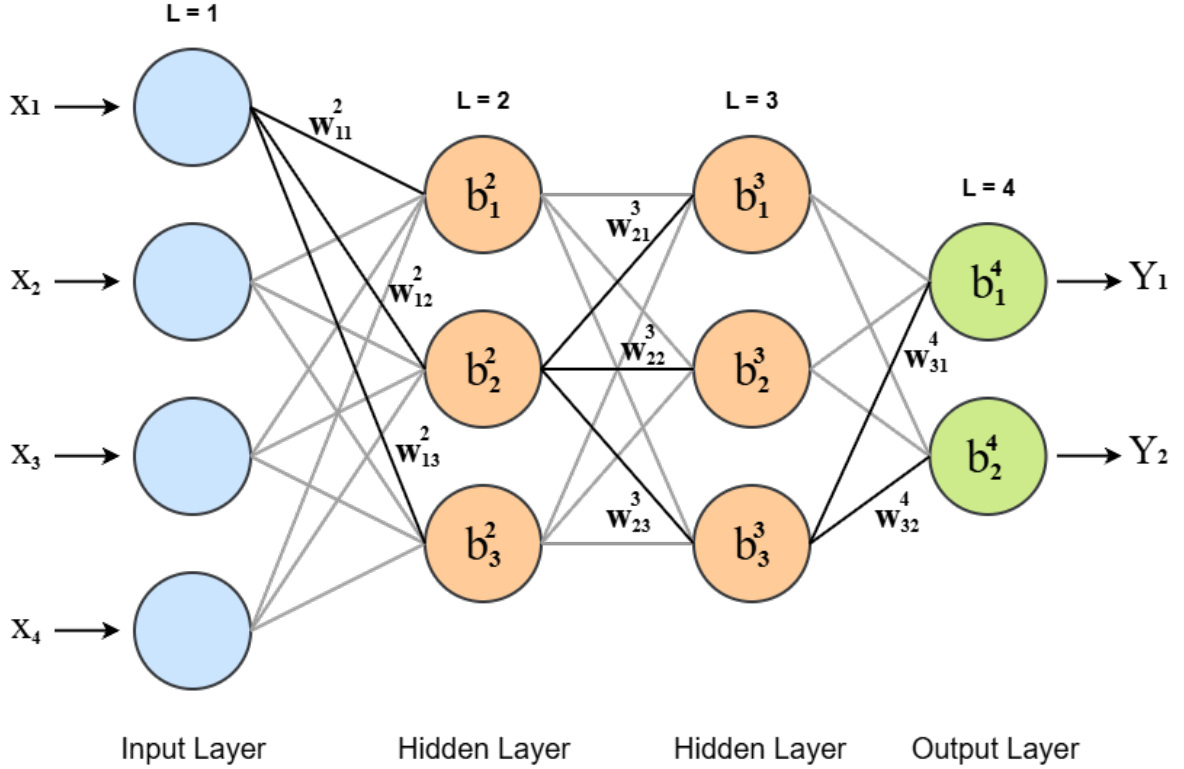


Figure 1.1: An example of Artificial Neural Network

To clarify how information propagates across the network, we first investigate the mathematical operation of a single neuron.

Consider a generic neuron i located in layer l . Naming $o_k^{(l-1)}$ the outputs of layer $l - 1$, the generic neuron sums all the $o_k^{(l-1)}$ multiplied by their corresponding weights and sums its bias. In math:

$$z_i^{(l)} = \sum_{k=1}^{N^{(l-1)}} w_{ki}^{(l)} o_k^{(l-1)} + b_i^{(l)} \quad (1.1)$$

where $N^{(l-1)}$ is the number of neurons in layer $l - 1$.

The intermediate value $z_i^{(l)}$ is commonly referred to as the *logit* and does not represent the final output of the neuron. Instead, the output is given by $o_i^{(l)} = \sigma(z_i^{(l)})$, where σ denotes an *activation function*. The activation function has two main purposes: it introduces non-linearity into the network, and it enables learning of complex data patterns. There are several types of activation functions, each characterized by different behaviors. The Rectified Linear Unit (ReLU) activation function, shown in Figure 1.2, is the main activation function employed in this work. It is defined by $ReLU(x) = \max(0, x)$, this means that if the input x is non-negative, ReLU returns x , if the input is negative, it returns 0. This behavior introduces sparsity in the network, which means that only a fraction of neurons activate at any given time, leading to more efficient and faster computation. Eventually, we can represent the step through layer l using matrix formalism.

$$\begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_n \end{bmatrix}_l = \begin{bmatrix} w_{11} & w_{12} & \dots & \dots & w_{1n} \\ w_{21} & w_{22} & \dots & \dots & w_{2n} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ w_{m1} & w_{m2} & \dots & \dots & w_{mn} \end{bmatrix}_l^T \begin{bmatrix} o_1 \\ o_2 \\ \vdots \\ o_m \end{bmatrix}_{l-1} + \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}_l \quad (1.2)$$

$$o_i^{(l)} = \sigma(z_i^{(l)}) = \sigma(w_{1i}^{(l)} o_1^{(l-1)} + w_{2i}^{(l)} o_2^{(l-1)} + \dots + w_{mi}^{(l)} o_m^{(l-1)} + b_i^{(l)}) \quad (1.3)$$

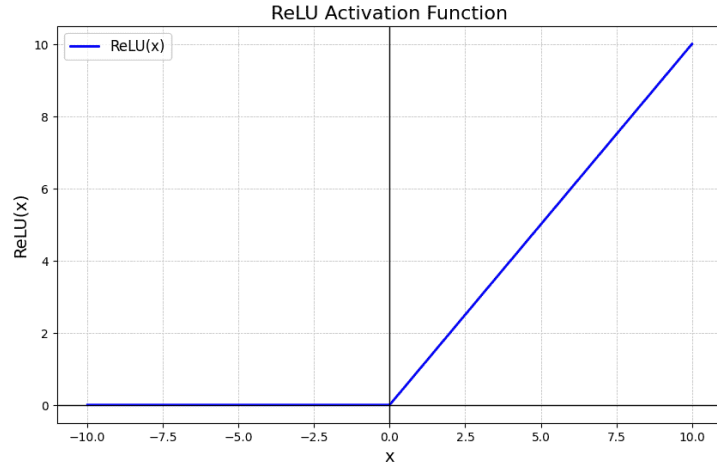


Figure 1.2: Rectified Linear Unit activation function

1.1.2 Neural Networks Learning Process

In Supervised Learning, the model receives the input data (or *features*) together with their corresponding labels. This allows us to define a *loss function* $C(y, \hat{y}(\vec{w}))$, also known as the *cost function*, which evaluates the error between the network's prediction $\hat{y}$ and the true label y . The learning process is driven by the *backpropagation* algorithm, which leverages the chain rule to compute the gradient of the cost function with respect to the parameters ($\vec{\nabla}_{\vec{w}} C$); subsequently, an optimization algorithm updates the parameters to minimize the loss. The most fundamental and common *optimizer* is first-order *gradient descent* [1, Chapter 4], which updates the parameters in the direction of steepest descent, though it is not the sole optimization technique. In practical applications, more advanced variants are widely preferred to accelerate convergence and escape local minima. These include Stochastic Gradient Descent (SGD) and the Adam (Adaptive Moment Estimation) optimizer, the latter of which dynamically scales the learning rate for individual parameters.

To illustrate the mathematical mechanism of backpropagation, consider a toy network where layer l consists of a single neuron, and the cost function defined as the mean squared error between the output ($o^{(l)}$) of layer l and the true label y . Here, n denotes the number of neurons in layer $l-1$, while the neuron index in layer l is omitted to ease the notation.

$$\begin{aligned}
 C(o^{(l)}, y) &= \frac{1}{2} (o^{(l)} - y)^2 \\
 o^{(l)} &= \sigma(z^{(l)}) = \sigma\left(\sum_{i=1}^n w_i^{(l)} o_i^{(l-1)} + b^{(l)}\right) \\
 \vec{\nabla} C &= \left(\frac{\partial C}{\partial w_1^{(l)}}, \dots, \frac{\partial C}{\partial w_n^{(l)}}, \frac{\partial C}{\partial b^{(l)}}\right) \\
 \frac{\partial C}{\partial w_i^{(l)}} &= \frac{\partial C}{\partial o^{(l)}} \frac{\partial o^{(l)}}{\partial z^{(l)}} \frac{\partial z^{(l)}}{\partial w_i^{(l)}} \\
 \frac{\partial C}{\partial b^{(l)}} &= \frac{\partial C}{\partial o^{(l)}} \frac{\partial o^{(l)}}{\partial z^{(l)}} \frac{\partial z^{(l)}}{\partial b^{(l)}}
 \end{aligned} \tag{1.4}$$

Once the gradient $\vec{\nabla} C$ has been calculated, the gradient descent algorithm determines the parameter update that maximizes the reduction of the cost function. By applying a first-order Taylor expansion, the change in the cost function ΔC resulting from a small variation of the parameters $\Delta \vec{w} = (\Delta w_1, \dots, \Delta w_n, \Delta b)^T$ can be expressed via the scalar [2]:

$$\Delta C \approx \vec{\nabla} C \cdot \Delta \vec{w} = \|\vec{\nabla} C\| \|\Delta \vec{w}\| \cos \theta \tag{1.5}$$

where θ represents the angle between the gradient vector $\vec{\nabla} C$ and the displacement vector $\Delta \vec{w}$.

To achieve the steepest descent, we want to make ΔC as negative as possible. This condition is fulfilled when $\cos \theta = -1$ ($\theta = \pi$), meaning that the parameter update must be in the opposite direction to the gradient. Consequently, we choose: $\Delta \vec{w} = -\eta \vec{\nabla} C$ where $\eta > 0$ is a hyperparameter known as the *learning rate*. The choice of this parameter is crucial; if it is too large, the algorithm fails to find the minimum; on the other hand, if it is too small, parameter optimization slows down, leading to unnecessarily long training times. A visual representation of this aspect is shown in Figure 1.3.

Extending this framework to a multi-layer network with numerous neurons significantly increases the number of trainable parameters, and consequently the dimensionality of the gradient of the loss function. The chain rule provides the mathematical basis for computing the partial derivative for each weight; the derivative of C with respect to a weight of the first layer is given by the product of several partial derivatives, since each layer depends on the previous one. However, computing each derivative independently is extremely inefficient.

The backpropagation algorithm resolves this computational bottleneck by expressing the chain rule as a sequence of matrix multiplications involving the *Jacobian matrices* of each consecutive layer [1, Chapter 6]. Moving backward from the output layer to the input, the algorithm propagates the gradient of the loss function by multiplying it with the Jacobian matrix of each layer. This matrix-based formulation optimizes computational performance and allows a quantum layer to be integrated into the network, as it simply needs to provide its own quantum Jacobian matrix.

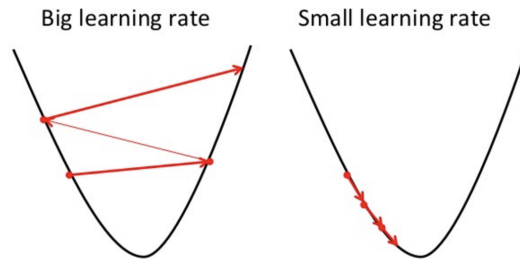


Figure 1.3: Visualization of how Learning Rate influences gradient descent

In a Machine Learning environment the *Backward pass* is the technical name for calculating the gradient, while the *Forward pass* refers to evaluating the data through the Neural Network model. Ultimately, the network can be trained through a *Training loop* which iterates the following steps: executing the forward pass, computing the gradient via the backward pass, and updating the parameters according to $w^{(t+1)} = w^{(t)} - \eta \vec{\nabla} C$. Each iteration is called *epoch*, usually represented by the letter t . The number of epochs in a training loop is another hyperparameter of the algorithm, which can affect its ability

to learn: if it is too small the model cannot learn properly causing *underfitting*; if it is too large, the model may lose generalization, causing *overfitting*, and the learning will be unnecessarily long.

1.1.3 Pytorch library

PyTorch [3] is an open-source deep learning framework developed in Python by the Facebook AI Research group. The base structure of this library is the *Tensor*, a multidimensional array similar to NumPy ndarrays [4], which contains a collection of numbers accessible via individual indices. A key feature of PyTorch tensors is their capability to be allocated on a Graphics Processing Unit (GPU), enabling parallel computations that are faster than those executed on a standard CPU.

The `torch.autograd` engine provides automatic differentiation, enabling the autonomous execution of the backpropagation algorithm. PyTorch tensors possess an essential property: they keep track of the computational graph and all operations that generated them, automatically evaluating the chain rule of derivatives. This behavior can be enabled during tensor instantiation by setting the attribute `requires_grad = True`:

```
params = torch.tensor([1.0, 1.0, ..., 1.0], requires_grad=True)
```

The values of the derivatives are stored directly within the `grad` attribute of the `params` tensor. Once the tensor is initialized, one evaluates the model output and the loss function, and then invoke the method `loss.backward()`; at this point, the `grad` attribute contains the partial derivatives of the loss function with respect to each parameter. A crucial detail regarding the `grad` attribute is that gradients are added rather than overwritten after calling `backward()`; therefore, it is necessary to zero out the gradients after each optimization step.

Another essential component of PyTorch is the `torch.optim` module, which implements various optimization algorithms to update parameters automatically based on the designated parameters and the specified learning rate. Even if the choice of an optimizer is flexible, in this work, the Adaptive Moment Estimation (**Adam**) optimizer is the only one used. Furthermore, PyTorch includes the `torch.nn` module dedicated to neural network design. It contains pre-built layers and modules to implement network architectures. This package also provides standard loss functions, such as Mean Squared Error (**MSELoss**) and Negative Log Likelihood Loss (**NLLLoss**), eliminating the need for manual implementation.

To implement a neural network in Python, it has to be defined as a custom Python class inheriting from `torch.nn.Module`. Neural architectures commonly utilize layers provided by `torch.nn`, such as linear layers, also known as *fully connected* layers, and convolutional layers. Linear layers behave as previously described, applying linear transformations with weights and biases. Convolutional layers are typically used for image classification,

because of their ability to filter the image into general characteristics instead of focusing on each pixel. Despite this, only linear layers have been used in this work; therefore, implementing convolutional layers could represent a promising direction to improve the network’s performance.

1.1.4 Image Recognition and the MNIST Dataset

Image classification, the task of categorizing a digital image into a predefined class, represents a fundamental application of neural networks. It belongs to the broader field of *computer vision*, where computational systems extract important patterns from digital images, videos, or other visual inputs to execute automated operations. The PyTorch module `torchvision.datasets` provides some default benchmark datasets, for instance the MNIST collection of handwritten digits (Figure 1.4) or CIFAR-10, which contains colored images of real-world objects. For the purposes of this work, it is worth clarifying how images are mapped into numerical tensors, since algorithms operate exclusively on tensor representations. A digital image is fundamentally a grid of pixels: a two-dimensional matrix of intensity values for grayscale images, or a three-dimensional tensor for images with three color channels, typically RGB images. By applying PyTorch transformations, such as `torchvision.transforms.ToTensor()`, raw images are converted into normalized tensors and subsequently fed into the neural network.

For classification tasks, the network’s final outputs represent a discrete probability distribution over all target classes. Consequently, each output result must lie between 0 and 1, with all class probabilities summing to unity. This is achieved by applying the *Softmax* [5] activation function to the output values z_i :

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}} \quad (1.6)$$

Eventually, to evaluate the predictive performance of the model, the dataset is divided into two distinct subsets: the training set and the validation set. The training set is employed exclusively during the optimization phase. Instead, the validation set consists of data which the model has never seen before, thereby avoiding *overfitting*. Overfitting happens when a model memorizes the training data instead of learning the general patterns; consequently, it fails to make accurate predictions on new, unseen validation data. This performance is quantified by defining the classification accuracy ($\mathcal{A}$) as the ratio of correctly predicted labels to the total number of evaluated samples:

$$\mathcal{A} = \frac{N_{\text{correct}}}{N_{\text{total}}} \quad (1.7)$$

expressed as a percentage.

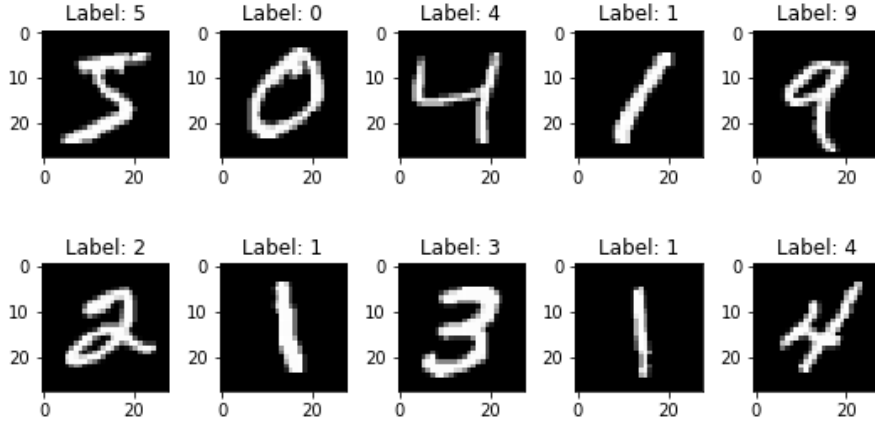


Figure 1.4: Examples of images from handwritten digits MNIST dataset

1.2 Quantum Machine Learning

Despite the remarkable success of classical Machine Learning, it faces significant limitations, primary among which is the high computational cost required to process large-scale or high-dimensional datasets. A promising alternative emerges from *Quantum Computing*, a paradigm that harnesses the fundamental principles of quantum mechanics to achieve computational advantages that surpass classical limits for specific classes of problem. Furthermore, quantum architectures are naturally suited to process quantum-native data. This intersection between artificial intelligence and quantum information processing has established the field of *Quantum Machine Learning* (QML) [6].

QML implementations can follow different approaches based on the nature of the data and algorithms, whether they are quantum or classical. In the current Noisy Intermediate-Scale Quantum (NISQ) era, quantum computers suffer from environmental noise and are prone to quantum decoherence, so a fully quantum approach is inefficient and impractical for large datasets; therefore, a combined framework is more advantageous. A primary example of this integrated approach is the *Hybrid Quantum-Classical Neural Network*, which forms the core focus of the following sections.

1.2.1 Introduction to Quantum Computing

This section aims to provide a brief introduction to the main features of quantum computing [7], in order to facilitate understanding of the topics that follow.

The quantum equivalent of the classical bit is the *qubit*, which is the fundamental unit of quantum information. It can be found in two states $|0\rangle$ and $|1\rangle$, as classical systems, but the fundamental principle of *quantum superposition* enables it to exist in a linear combination of both states simultaneously. A general single-qubit state $|\psi\rangle$ can be

written as:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle \quad (1.8)$$

where $\alpha, \beta \in \mathbb{C}$ are complex probability amplitudes. Single qubits correspond to vectors from a two-dimensional complex Hilbert space $\mathcal{H}_2$, spanned by the computational basis $\{|0\rangle, |1\rangle\}$. The squared module of the parameters α and β give the probabilities of measuring the qubit in state $|0\rangle$ or $|1\rangle$, respectively, so they have to satisfy the normalization condition:

$$|\alpha|^2 + |\beta|^2 = 1 \quad (1.9)$$

When a measurement is performed in the computational basis, the superposition collapses into a single basis state, $|0\rangle$ or $|1\rangle$. This measurement process converts the probabilistic quantum state into a classical outcome, projecting the system out of its superposition. The reason for the increasing interest in quantum computing becomes more apparent when considering multi-qubit systems, where superposition gives rise to state space expansion. For instance, a two-qubit state is represented as a linear combination of four different states:

$$|\psi\rangle_{q_1 q_2} = \alpha_{00}|00\rangle + \alpha_{01}|01\rangle + \alpha_{10}|10\rangle + \alpha_{11}|11\rangle \quad (1.10)$$

By applying a single quantum operation to $|\psi\rangle$, all four components are transformed simultaneously, whereas a classical system would represent the same information with four different states, and would have to compute four separate operations to achieve the same transformation as the quantum system.

A geometric visualization of a single-qubit state is provided by the *Bloch sphere* (Figure 1.5), which consists of a three-dimensional spherical surface with unit radius. Any pure qubit state can be parameterized as a vector on the Bloch sphere using two real angles $\theta \in [0, \pi]$ and $\phi \in [0, 2\pi)$:

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle + e^{i\phi}\sin\left(\frac{\theta}{2}\right)|1\rangle \quad (1.11)$$

This geometric framework is particularly useful for describing quantum operations, commonly referred to as *quantum gates*. Indeed, while single-qubit gates in a two-dimensional Hilbert space are represented by 2×2 unitary matrices U , these operations can be interpreted as rotations of the state vector on the Bloch sphere around a given axis by an angle θ :

$$R_x(\theta) = \begin{pmatrix} \cos \frac{\theta}{2} & -i \sin \frac{\theta}{2} \\ -i \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{pmatrix}, \quad R_y(\theta) = \begin{pmatrix} \cos \frac{\theta}{2} & -\sin \frac{\theta}{2} \\ \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{pmatrix}, \quad R_z(\theta) = \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix} \quad (1.12)$$

It can be shown via Euler angle decomposition that any single-qubit unitary transformation U can be factorized, up to a global phase ($e^{i\gamma}$), as a product of three fundamental rotations around orthogonal axes:

$$U = e^{i\gamma} R_z(\alpha) R_y(\beta) R_z(\delta) \quad (1.13)$$

with $\alpha, \beta, \delta, \gamma \in R$. Achieving single-qubit universality is fundamental in quantum computing, because it enables the construction of transformations that map a state vector to any arbitrary state, spanning the entire surface of the Bloch sphere. Rotation gates are a good example of *parametrized gates*, since they depend on the value of θ . In general, parametrized operations can depend on one or more variables; since neural networks learn through parameter optimization, this category of gate will be crucial for a hybrid neural network implementation.

The *Hadamard* gate and the family of *Pauli* gates are examples of unitary single qubit transformation, so they can be factorized via Euler decomposition. There are three Pauli gates:

- the X gate = $\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ swaps the computational basis states (it is the quantum equivalent of a classical NOT gate) and it can be expressed as a rotation $R_x(\pi)$;
- the Y gate = $\begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$ swaps the computational basis states but with a complex phase factor and it can be expressed as a rotation $R_y(\pi)$;
- the Z gate = $\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$ applies a phase flip to the quantum state and it can be expressed as a rotation $R_z(\pi)$.

Pauli matrices are self-inverse, and their product (up to a phase factor) returns the identity matrix:

$$I^2 = X^2 = Y^2 = Z^2 = -iXYZ = I \quad (1.14)$$

The Hadamard gate is a special quantum gate that creates an equal superposition of the computational basis:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad \text{such that} \quad H|0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \quad H|1\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}} \quad (1.15)$$

It can be expressed as a 180 degrees rotation around x-axis followed by a 90 degrees rotation around y-axis.

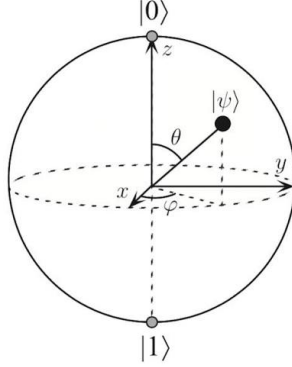


Figure 1.5: Geometric visualization of a single-qubit state on the Bloch sphere

1.2.2 Qudits and High-Dimensional Hilbert Spaces

Having established the standard qubit framework, we now extend our computational building blocks to higher dimensions. This mathematical generalization is necessary to describe the multi-level quantum states implemented in our hybrid network. Therefore, we introduce qudits as the natural d -dimensional generalization of the conventional two-level qubits [8].

While a qubit is defined in a two-dimensional Hilbert space $\mathcal{H}_2$, the state of a single qudit is described by a vector in a d -dimensional complex Hilbert space $\mathcal{H}_d$, spanned by a set of orthonormal basis vectors $\{|0\rangle, |1\rangle, \dots, |d-1\rangle\}$. The state of a qudit has the general form:

$$|\psi\rangle_d = \alpha_0|0\rangle + \alpha_1|1\rangle + \dots + \alpha_{d-1}|d-1\rangle = \sum_{l=0}^{d-1} \alpha_l |l\rangle \quad (1.16)$$

where the complex coefficients $\alpha_l \in \mathbb{C}$ represent the probability amplitudes and must satisfy the generalized normalization constraint:

$$\sum_{l=0}^{d-1} |\alpha_l|^2 = 1 \quad (1.17)$$

The measurement process follows the standard postulates of quantum mechanics. Performing a projective measurement in the computational basis collapses the qudit's wave function into a specific eigenstate $|l\rangle$ with a probability given by:

$$P(|l\rangle) = |\alpha_l|^2 \quad (1.18)$$

Finally, a major distinction arises regarding geometric representation. While a single qubit can be intuitively mapped onto the surface of the three-dimensional Bloch sphere,

no such straightforward geometric representation exists for qudits ($d > 2$). This high-dimensional state space makes the mathematical description and visualization of qudit states significantly more complex, requiring abstract algebraic frameworks rather than intuitive physical models.

The mathematical formulation and implementation of d -dimensional qudit gates are deferred to Chapter 2.

1.2.3 Cirq library

Cirq is a Python software library [9] for writing, manipulating, and optimizing quantum circuits, and then running them on quantum computers and quantum simulators.

It provides several default modules to define qubits, construct circuits, and perform simulations; in just a few steps, it is possible to execute a complete quantum circuit, as shown in Figure 1.6. Cirq library also provides the `cirq.Qid` class, which generalizes the standard two-level `cirq.Qubit` to represent high-dimensional quantum systems of arbitrary dimension d . This class enables the initialization of both qubits and qudits, since d can be arbitrarily set to 2 to recover the standard qubit representation. Although the library is natively compatible with high-dimensional systems, it lacks built-in definitions for qudit gates; consequently, they must be constructed from scratch (this will be addressed in the next section).

```
import cirq

# Pick a qubit.
qubit = cirq.GridQubit(0, 0)

# Create a circuit
circuit = cirq.Circuit(
    cirq.X(qubit)**0.5, # Square root of NOT.
    cirq.measure(qubit, key='m') # Measurement.
)
print("Circuit:")
print(circuit)

# Simulate the circuit several times.
simulator = cirq.Simulator()
result = simulator.run(circuit, repetitions=20)
print("Results:")
print(result)
```

Figure 1.6: Example of a quantum circuit implemented using the Cirq library.

1.2.4 Hybrid Neural Networks

In this project, PyTorch and Cirq libraries have been used to code the machine learning structure and the quantum component, respectively. Our network has been designed to face a classification problem, using handwritten digits from the MNIST dataset as input data.

Directly processing high-dimensional raw data would require a large number of qubits; however, simulating a large number of qubits on classical hardware, or physically fabricating them, poses severe technological and computational constraints. To overcome this bottleneck, classical hidden layers can be employed to compress the input features into a lower-dimensional representation. Since the primary objective of this work is to test and evaluate the expressive power of a single-qudit circuit, a hybrid quantum-classical architecture [10, Chapter 6] is adopted. It can be implemented by embedding a parametrized quantum circuit ($QC(\vec{\theta})$) among other classical layers. The main architecture remains the same; the classical hidden layers process the input data as discussed in Section 1.1.1 until the information reaches the quantum layer (l_Q). The quantum forward pass consists of resolving the parameters of the circuit ($\vec{\theta}$) and then computing a statistical measurement. Specifically, resolving the parameters means that the outputs of the previous layer are assigned to the gates' rotation angle, thereby preparing the circuit for execution:

$$\theta_i^{(l_Q)} = o_i^{(l_Q-1)}(z_i^{l_Q-1}) \quad (1.19)$$

where the index i runs from 1 to N , with N being the total number of parameters. Furthermore, performing a statistical measurement on the qudit involves executing the circuit for a fixed number of repetitions (*shots*) to count how many times the quantum system collapses into each of its computational states, reconstructing the output probability distribution by normalization. The results of this measurement constitute the output of the quantum layer, which can either serve as input for subsequent layers or act as the final prediction of the network.

A schematic representation of this pipeline is shown in Figure 1.7.

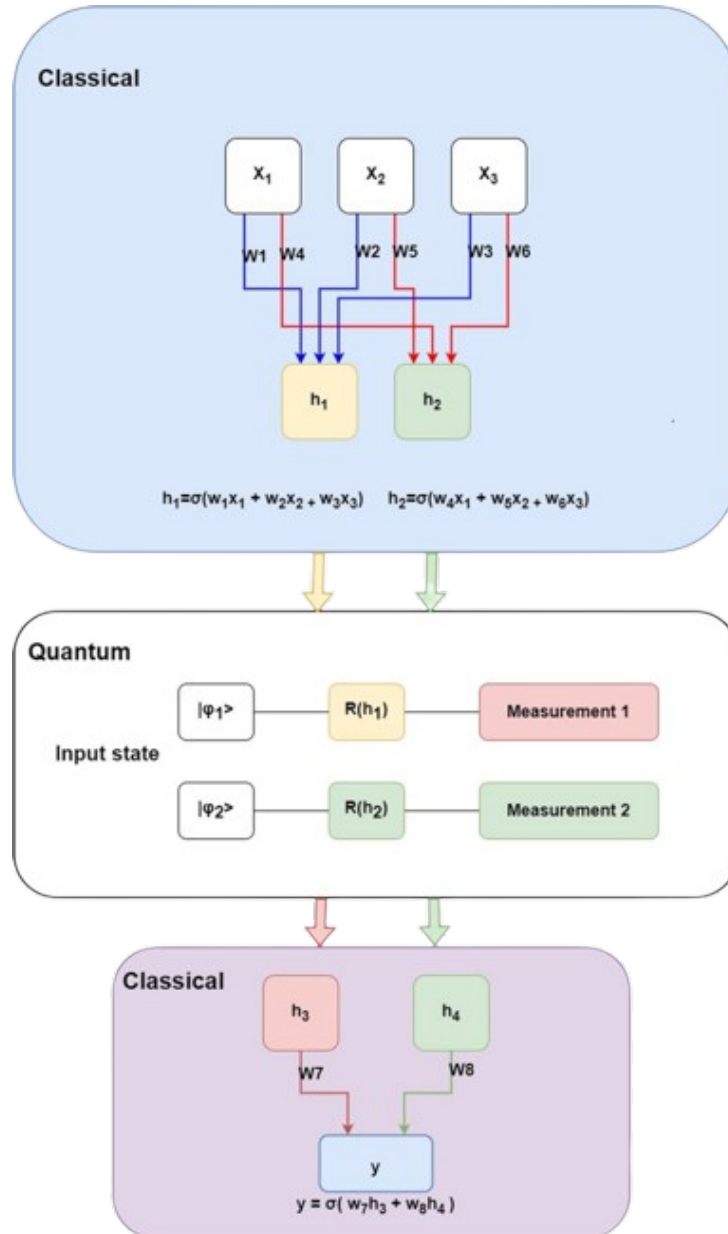


Figure 1.7: Scheme of a quantum classical neural network

1.2.5 Backpropagation with quantum layers

The backward pass for a quantum layer is implemented by using the *parameter shift rule* [11] to define a quantum operation gradient of the l_Q layer, with respect to the parameters of the circuit ($\vec{\nabla}_{\vec{\theta}} QC(\vec{\theta})$). The backpropagation algorithm reduces the computation of all gradients to a product of matrices, and this modularity allow us to integrate a quantum layer by multiplying $\vec{\nabla}_{\vec{\theta}} QC(\vec{\theta})$ by the gradients of previous layers. A brief explanation of the parameter shift rule is provided below.

Firstly, we shift all the parameters by a fixed value $\vec{s}$ obtaining two other sets of parameters:

$$\vec{\theta}_{\pm} = \vec{\theta} \pm \vec{s} \quad (1.20)$$

then, we compute the gradient in the following way:

$$\vec{\nabla}_{\vec{\theta}} QC(\vec{\theta}) = QC(\vec{\theta}_+) - QC(\vec{\theta}_-) \quad (1.21)$$

where $QC(\vec{\theta})$ represent the combination of all quantum operations parametrized by the vector of parameters $\vec{\theta}$. The value of the shift is not arbitrarily small, as someone could assume from classical derivatives, but it is different for each quantum operation. It can be shown that for rotation gates $s = \pi/2$.

Chapter 2

Qudit-Based Neural Networks

This section presents the core of this thesis, focusing on the physical characteristics of d -level quantum systems (qudits) and their key differences from standard two-level qubits. To explore these systems, we designed specific quantum circuits and integrated them as hybrid layers within a neural network architecture. In addition to comparing their final performance, we discuss the essential steps and design choices required to implement and optimize these models.

2.1 Introduction to qudits

An increase in the dimensionality of the Hilbert space allows each fundamental computational unit to encode a larger amount of information; consequently, a single qudit can store more information than a standard two-level qubit. When extending this framework to multiple physical units, a fundamental difference in system scalability emerges. An n -qubit register maps onto a Hilbert space of dimension 2^n , whereas an n -qudit system with local dimension d spans a Hilbert space of dimension d^n [12]. This exponential growth enables what is known as *vertical scaling*: by increasing the local dimensionality d rather than the number of physical carriers n (*horizontal scaling*), it is possible to store a higher information density and implement significantly more compact and efficient quantum circuits.

Replacing qubits with qudits in a quantum circuit inevitably reduces the required number of inter-qudit gates between different carriers, while single-qudit operations increase. Whether this trade-off is advantageous depends on the specific hardware platform and the type of gate used; however, inter-qudit gates are typically the most susceptible to environmental noise and decoherence, so the reduction in two-body gate count can enhance the overall circuit fidelity [12].

However, this high-dimensional scaling introduces a fundamental practical trade-off: the control complexity scales rapidly with the local dimension d . While a qubit requires

control over a single transition, a qudit generally involves multiple closely spaced transitions, often characterized by non-uniform anharmonicity and strict selection rules. As d increases, these additional energy levels can introduce, for some hardware implementations, new and more complex decoherence pathways.

2.1.1 Universality condition

Beyond the physical advantages and engineering complexities, a fundamental theoretical prerequisite is the formulation of a universal gate set. Exact universality implies that any arbitrary unitary transformation can be simulated with arbitrary precision or zero error. A physically viable decomposition scheme was proposed by Muthukrishnan and Stroud [13], who demonstrated that any N -qudit unitary operation can be decomposed using a combination of single- and two-qudit gates. Following this paradigm, Brennen et al. [14] established rigorous criteria for exact qudit universality by leveraging the QR decomposition of unitary matrices. They proved that universal control is achievable using a set of single-qudit operations paired with a single-parameter controlled-phase gate, provided that the physical couplings between the basis states form a connected coupling graph (meaning that any state can be transitioned into any other through a sequence of directly driven transitions).

In the context of quantum machine learning, this requirement of a connected graph translates directly into the necessity of coupling the qudit’s energy levels, as the variational quantum layer must be capable of driving transitions and transferring population between arbitrary d -level states. In practice, this global coupling is achieved by decomposing the system’s evolution into a sequence of operations acting on two-dimensional subspaces. By defining parameterized rotation gates that target specific pairs of energy levels $\{|i\rangle, |j\rangle\}$, we can chain these localized couplings to satisfy the connectivity criteria required for universality.

While theoretical universality is a fundamental requirement, physical feasibility remains a concern. It is worth noting that our circuit is implemented on an idealized quantum simulator and is not tailored for a specific physical platform; consequently, the gates introduced in the following section may not fully account for the practical constraints and physical limitations of current quantum hardware.

2.1.2 Generalized Gates

It must be emphasized that the gate set introduced in this section is not specifically optimized for the classification task at hand. Instead, these operators were selected as a pragmatic starting point to satisfy the connected coupling graph requirement using straightforward, intuitive extensions of standard two-dimensional gates. Hence, a deeper

investigation into alternative circuit layouts and gate selections is required to identify the most effective architecture for this class of problems.

Since any two-dimensional subspace behaves as an isolated two-dimensional Hilbert space, we can naturally extend the qubit rotation gates to act selectively on any chosen pair of energy levels $\{|i\rangle, |j\rangle\}$. Specifically, these gates drive coherent transitions and phase shifts exclusively within the $\{|i\rangle, |j\rangle\}$ subspace, while leaving all other computational basis states untouched, acting as the identity on the remaining subspace [14].

First, we introduce the generalized Givens rotation $R_{ij}(\theta, \phi)$ [15], which serves as a two-parameter variational gate. Second, we establish three subspace-restricted rotation gates, designated as $X_{ij}(\theta)$, $Y_{ij}(\theta)$, and $Z_{ij}(\phi)$. This nomenclature stems from the fact that they directly generalize the standard single-qubit $R_x(\theta)$, $R_y(\theta)$, and $R_z(\phi)$ rotations (see Section 1.2.1), preserving the algebraic structure of the Pauli matrices within the targeted two-dimensional subspace. When applying these operators, the parameter θ drives the state population by mixing the probability amplitudes of the target states, whereas the parameter ϕ introduces a relative quantum phase.

Finally, we incorporate the generalized Hadamard gate H_d , which is a static, non-parameterized gate, employed to create an equal superposition of all d computational basis states.

Variational Givens Rotation Gate (R_{ij})

The gate $R_{ij}(\theta, \phi)$ represents the most general parameterized rotation within the targeted $\{|i\rangle, |j\rangle\}$ subspace, acting as a coherent coupling where the angle θ drives the population transfer and ϕ introduces a relative quantum phase. In the original paper [15], this gate is introduced in its operator form as $\hat{R}_{ij}(\theta, \phi) = \exp\left(-i\frac{\theta}{2}\hat{G}_{ij}(\phi)\right)$, where $\hat{G}_{ij}(\phi) = \cos\phi\hat{\sigma}_{ij}^x - \sin\phi\hat{\sigma}_{ij}^y$; however, to implement this operator in our simulation framework, we explicitly compute its matrix representation:

$$R_{ij}(\theta, \phi) = \begin{matrix} & |i\rangle & & & |j\rangle & & \\ \begin{matrix} |i\rangle \\ |j\rangle \end{matrix} & \begin{pmatrix} 1 & \dots & 0 & \dots & 0 & \dots & 0 \\ \vdots & \ddots & \vdots & & \vdots & & \vdots \\ 0 & \dots & \cos(\theta/2) & \dots & -ie^{i\phi}\sin(\theta/2) & \dots & 0 \\ \vdots & & \vdots & \ddots & \vdots & & \vdots \\ 0 & \dots & -ie^{-i\phi}\sin(\theta/2) & \dots & \cos(\theta/2) & \dots & 0 \\ \vdots & & \vdots & & \vdots & \ddots & \vdots \\ 0 & \dots & 0 & \dots & 0 & \dots & 1 \end{pmatrix} \end{matrix} \quad (2.1)$$

Two-Level X-Rotation Gate (X_{ij})

The gate $X_{i,j}(\theta)$ introduces a complex coupling with a $\pi/2$ phase shift ($-i = e^{i\pi/2}$) between the $|i\rangle$ and $|j\rangle$ levels, representing a rotation about the local X -axis of the subspace:

$$X_{ij}(\theta) = \begin{array}{cc} & \begin{array}{cc} |i\rangle & |j\rangle \end{array} \\ \begin{array}{c} |i\rangle \\ |j\rangle \end{array} & \begin{pmatrix} 1 & \dots & 0 & \dots & 0 & \dots & 0 \\ \vdots & \ddots & \vdots & & \vdots & & \vdots \\ 0 & \dots & \cos(\theta/2) & \dots & -i \sin(\theta/2) & \dots & 0 \\ \vdots & & \vdots & \ddots & \vdots & & \vdots \\ 0 & \dots & -i \sin(\theta/2) & \dots & \cos(\theta/2) & \dots & 0 \\ \vdots & & \vdots & & \vdots & \ddots & \vdots \\ 0 & \dots & 0 & \dots & 0 & \dots & 1 \end{pmatrix} \end{array} \quad (2.2)$$

Two-Level Y-Rotation Gate (Y_{ij})

The gate $Y_{i,j}(\theta)$ performs a real-valued rotation coupling the states $|i\rangle$ and $|j\rangle$, representing a rotation about the local Y -axis of the subspace:

$$Y_{ij}(\theta) = \begin{array}{cc} & \begin{array}{cc} |i\rangle & |j\rangle \end{array} \\ \begin{array}{c} |i\rangle \\ |j\rangle \end{array} & \begin{pmatrix} 1 & \dots & 0 & \dots & 0 & \dots & 0 \\ \vdots & \ddots & \vdots & & \vdots & & \vdots \\ 0 & \dots & \cos(\theta/2) & \dots & -\sin(\theta/2) & \dots & 0 \\ \vdots & & \vdots & \ddots & \vdots & & \vdots \\ 0 & \dots & \sin(\theta/2) & \dots & \cos(\theta/2) & \dots & 0 \\ \vdots & & \vdots & & \vdots & \ddots & \vdots \\ 0 & \dots & 0 & \dots & 0 & \dots & 1 \end{pmatrix} \end{array} \quad (2.3)$$

Two-Level Phase Rotation Gate (Z_{ij})

The gate $Z_{i,j}(\phi)$ applies a relative phase shift of ϕ within the $\{|i\rangle, |j\rangle\}$ subspace, representing a rotation about the local Z -axis of the subspace :

$$\begin{aligned}
Z_{ij}(\phi) = & \begin{array}{cc} & \begin{array}{cc} |i\rangle & |j\rangle \end{array} \\ \begin{array}{c} |i\rangle \\ |j\rangle \end{array} & \begin{pmatrix} 1 & \dots & 0 & \dots & 0 & \dots & 0 \\ \vdots & \ddots & \vdots & & \vdots & & \vdots \\ 0 & \dots & e^{-i\phi/2} & \dots & 0 & \dots & 0 \\ \vdots & & \vdots & \ddots & \vdots & & \vdots \\ 0 & \dots & 0 & \dots & e^{i\phi/2} & \dots & 0 \\ \vdots & & \vdots & & \vdots & \ddots & \vdots \\ 0 & \dots & 0 & \dots & 0 & \dots & 1 \end{pmatrix} \end{array}
\end{aligned} \tag{2.4}$$

Generalized d -Dimensional Hadamard Gate (H_d)

Following the formulation proposed by Pudda et al. [16], the generalized Hadamard gate H_d maps the computational basis states into a fully balanced coherent superposition across the entire d -dimensional space, and its matrix representation is given by:

$$H_d = \frac{1}{\sqrt{d}} \begin{pmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \omega^{d-1} & \omega^{2(d-1)} & \dots & \omega^{(d-1)^2} \\ 1 & \omega^{d-2} & \omega^{2(d-2)} & \dots & \omega^{(d-1)(d-2)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega & \omega^2 & \dots & \omega^{(d-1)} \end{pmatrix}, \text{ with } \omega = e^{i\frac{2\pi}{d}} \tag{2.5}$$

2.2 Qudit Neural Network implementation

Having established the mathematical framework, we now have the necessary tools to construct a functional variational qudit circuit tailored for quantum machine learning. A primary requirement for any quantum neural network is to guarantee trainability. Not all gate configurations are suitable for gradient-based optimization; specifically, the computational energy levels must be coupled to one another by parameterized operations. In the absence of these active transition pathways, the gradient descent algorithm cannot update the probability amplitudes of the isolated states, leading to vanishing gradients. To prevent this, we utilize sequences of parameterized gates that sequentially connect each energy level to the next, thereby establishing a full parametric connection across all d computational states. Consider the initial state $|\psi_0\rangle$ in a three-dimensional Hilbert space:

$$|\psi_0\rangle = |0\rangle = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} \quad (2.6)$$

We then apply $X_{0,1}(\theta_1)$ to the subspace $\{0, 1\}$ which couples the states $|0\rangle$ and $|1\rangle$:

$$|\psi_1\rangle = \hat{X}_{0,1}(\theta_1)|\psi_0\rangle = \begin{pmatrix} \cos \frac{\theta_1}{2} & -i \sin \frac{\theta_1}{2} & 0 \\ -i \sin \frac{\theta_1}{2} & \cos \frac{\theta_1}{2} & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} \cos \frac{\theta_1}{2} \\ -i \sin \frac{\theta_1}{2} \\ 0 \end{pmatrix} \quad (2.7)$$

This yields the state $|\psi_1\rangle = \cos \frac{\theta_1}{2}|0\rangle - i \sin \frac{\theta_1}{2}|1\rangle$, meaning that the parameter θ_1 directly controls the probability amplitudes of these two states. Subsequently, to achieve connectivity with the remaining computational state, we apply $X_{1,2}(\theta_2)$ to the subspace $\{1, 2\}$:

$$|\psi_2\rangle = \hat{X}_{1,2}(\theta_2)|\psi_1\rangle = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \frac{\theta_2}{2} & -i \sin \frac{\theta_2}{2} \\ 0 & -i \sin \frac{\theta_2}{2} & \cos \frac{\theta_2}{2} \end{pmatrix} \begin{pmatrix} \cos \frac{\theta_1}{2} \\ -i \sin \frac{\theta_1}{2} \\ 0 \end{pmatrix} \quad (2.8)$$

$$|\psi_2\rangle = \begin{pmatrix} \cos \frac{\theta_1}{2} \\ 0 + \cos \frac{\theta_2}{2} (-i \sin \frac{\theta_1}{2}) + 0 \\ 0 + (-i \sin \frac{\theta_2}{2}) (-i \sin \frac{\theta_1}{2}) + 0 \end{pmatrix} = \begin{pmatrix} \cos \frac{\theta_1}{2} \\ -i \sin \frac{\theta_1}{2} \cos \frac{\theta_2}{2} \\ -\sin \frac{\theta_1}{2} \sin \frac{\theta_2}{2} \end{pmatrix} \quad (2.9)$$

These two parameters are now able to transfer population among all three states, resulting in parameterized measurement probabilities:

$$P(0) = \cos^2 \frac{\theta_1}{2}, \quad P(1) = \sin^2 \frac{\theta_1}{2} \cos^2 \frac{\theta_2}{2}, \quad P(2) = \sin^2 \frac{\theta_1}{2} \sin^2 \frac{\theta_2}{2} \quad (2.10)$$

2.2.1 Initial State Preparation

Another critical aspect to consider is the choice of the initial state. If, for instance, we apply a parameterized unitary layer consisting of a combination of generalized Givens rotation ($R^{(k)}(\theta_k, \phi_k)$) directly to a computational basis state, the phase parameters ϕ_k completely vanish upon measurement. Consequently, the classical optimizer remains blind to these parameters, resulting in a loss of expressive power and reduced training efficiency.

To illustrate this in the simplest case, we consider a single qubit system (the same reasoning can be applied to $d > 2$). The parameterized rotation gate $R(\theta, \phi)$ in this Hilbert space is defined as:

$$R(\theta, \phi) = \begin{pmatrix} \cos \frac{\theta}{2} & -ie^{i\phi} \sin \frac{\theta}{2} \\ -ie^{-i\phi} \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{pmatrix} \quad (2.11)$$

If we apply the gate directly to the ground state $|\psi_0\rangle = |0\rangle = \begin{pmatrix} 1 & 0 \end{pmatrix}^T$, the obtained state is:

$$|\psi_1\rangle = \hat{R}(\theta, \phi) \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} \cos \frac{\theta}{2} \\ -ie^{-i\phi} \sin \frac{\theta}{2} \end{pmatrix} \quad (2.12)$$

The resulting measurement probabilities are:

$$P(0) = \left| \cos \frac{\theta}{2} \right|^2 = \cos^2 \frac{\theta}{2} \quad (2.13)$$

$$P(1) = \left| -ie^{-i\phi} \sin \frac{\theta}{2} \right|^2 = \sin^2 \frac{\theta}{2} \quad (2.14)$$

Because $|e^{-i\phi}|^2 = 1$, the phase parameter ϕ completely vanishes. The gradient with respect to ϕ is zero ($\partial P(k)/\partial \phi = 0$), meaning the network cannot train this parameter.

Conversely, preparing the qudit in a coherent superposition of states via a generalized Hadamard gate (H_d) ensures that the relative phase ϕ is preserved through quantum interference, allowing it to actively modulate the final measurement probabilities [15]. Therefore, we first apply a Hadamard gate $H_{d=2}$, to prepare the qubit in the equal superposition state $|\psi_H\rangle = H_{d=2}|\psi_0\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \end{pmatrix}^T$, followed by the rotation:

$$|\psi_1\rangle = R(\theta, \phi)|\psi_H\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} \cos \frac{\theta}{2} - ie^{i\phi} \sin \frac{\theta}{2} \\ -ie^{-i\phi} \sin \frac{\theta}{2} + \cos \frac{\theta}{2} \end{pmatrix} \quad (2.15)$$

Therefore, the final measurement probabilities are:

$$P(0) = \frac{1}{2} \left| \cos \frac{\theta}{2} - ie^{i\phi} \sin \frac{\theta}{2} \right|^2 = \dots = \frac{1 + \sin \theta \sin \phi}{2} \quad (2.16)$$

$$P(1) = \frac{1}{2} \left| -ie^{-i\phi} \sin \frac{\theta}{2} + \cos \frac{\theta}{2} \right|^2 = \dots = \frac{1 - \sin \theta \sin \phi}{2} \quad (2.17)$$

Due to the interference term $\sin \theta \sin \phi$, the phase ϕ is preserved. The gradient is now non-zero ($\partial P(0)/\partial \phi = \frac{1}{2} \sin \theta \cos \phi \neq 0$), enabling successful gradient-based optimization.

While in some configurations this phase-canceling represents only a minor performance bottleneck, and the network can still converge using only the population transfer parameter θ , in other architectures the initial superposition is mathematically indispensable. For instance, in circuits composed of phase-shift gates (such as $Z_{ij}(\phi)$ gates) which only alter relative phases, the network is entirely unable to train without an initial H_d gate. Specifically, in these architectures, a final H_d gate is also strictly required right before measurement to prevent the phase from vanishing, otherwise the network degenerates into random guessing.

2.2.2 Qudit and Custom Gate Implementation in Cirq

Having established the theoretical framework of qudit gates, we now focus on their practical realization within Google Cirq. As mentioned in Section 1.2.3, Cirq does not provide native gates for qudit architectures; consequently, custom qudit gates must be constructed by defining Python classes that inherit from the `cirq.Gate` class [17].

To implement a functional qudit gate from scratch, we have to define three fundamental Cirq protocols (internal methods designated by leading and trailing underscores):

- `_qid_shape_`: defines the dimension of the target quantum systems. For a single qudit of dimension d , it returns a tuple $(d,)$, ensuring that the gate is mathematically represented as a square $d \times d$ matrix;
- `_unitary_`: computes and returns the matrix representation of the unitary operator;
- `_circuit_diagram_info_`: specifies the text-based label displayed when printing the circuit.

While these three protocols are sufficient for static operations, constructing a parameterized quantum neural network layer requires integrating symbolic variables. In our architecture, we employ the `SymPy` library to define algebraic variables representing the variational parameters. To dynamically map the numerical outputs of the preceding classical layer to these symbolic placeholders, Cirq provides a parameter resolver (`cirq.ParamResolver` [18]), which is natively integrated into the simulator’s execution methods (such as `simulate()` or `run()`). This resolver acts as the key interface between the classical and quantum layers: it ingests the float values generated by the network and maps each numerical value to its corresponding symbolic parameter in the quantum circuit prior to simulation.

Because standard Python numerical operations cannot directly evaluate symbolic placeholders, the custom gate class must be designed to handle non-numerical inputs

during the circuit construction phase. This is achieved by implementing three additional protocols:

- `_is_parameterized_`: checks if the gate contains variables instead of concrete numbers. if variables are present, it signals the simulator to wait for their actual values before attempting to calculate the gate's matrix;
- `_parameter_names_`: identifies and returns a set containing the names of all free symbolic variables within the gate;
- `_resolve_parameters_`: receives the `cirq.ParamResolver` as an argument, replaces the symbolic variables with their resolved numerical values, and returns a new, non-parameterized instance of the gate ready for simulation.

Figure 2.1 illustrates a complete, functional Python class showcasing the integration of both representation and parameter-resolution protocols for a custom qudit gate.

```

import cirq
from cirq import Gate
import sympy
from typing import Union
import numpy as np

# Rotation gate about the local X-axis of the subspace {i,j}
class X_ij(Gate):
    def __init__(self, i, j, phase : Union[float, sympy.Symbol] , dimension=3, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.i = i
        self.j = j
        self.d = dimension
        self.phase = phase

    def _qid_shape_(self):
        return (self.d,)

    def _unitary_(self):
        matrix = np.eye(self.d, dtype=complex)
        matrix[self.i][self.i] = matrix[self.j][self.j] = np.cos(self.phase / 2) # type: ignore
        matrix[self.j][self.i] = matrix[self.i][self.j] = -1j * np.sin(self.phase / 2) # type: ignore
        return matrix

    def _circuit_diagram_info_(self, args):
        return f"X({self.phase})_{self.i}{self.j}"

    def _is_parameterized_(self) -> bool:
        return isinstance(self.phase, sympy.Basic)

    def _parameter_names_(self):
        if isinstance(self.phase, sympy.Symbol):
            return (self.phase.name,)
        elif isinstance(self.phase, sympy.Basic):
            return tuple(str(param) for param in self.phase.free_symbols)
        return ()

    def _resolve_parameters_(self, resolver, recursive):
        resolved_phase = resolver.value_of(self.phase, recursive)
        return X_ij(self.i, self.j, dimension=self.d, phase=resolved_phase)

# Here we create a qutrit for the gate to act on.
q0 = cirq.LineQid(0, dimension=3)

# We can now enact the gate on this qutrit.
circuit = cirq.Circuit(X_ij(0,1,np.pi,dimension=3).on(q0))

```

Figure 2.1: Cirq implementation of the parameterized qudit gate X_{ij} . The code illustrates also the initialization of a single qutrit ($d = 3$) via the `cirq.LineQid` class and the subsequent application of the custom operator on it using the `.on()` method.

2.2.3 Loss function

In this work, two widely used cost functions for classification tasks are considered: Cross-Entropy Loss (`CrossEntropyLoss`) [19] and Negative Log-Likelihood Loss (`NLLLoss`) [20]. Although these loss functions are mathematically equivalent when applied to the same probability distribution, they expect inputs in different formats. In our training loop we only implemented `NLLLoss`, but both are analyzed here to justify this architectural choice.

The `CrossEntropyLoss` function expects raw, unnormalized input data (*logits*). This is because PyTorch internally applies a `LogSoftmax` [21] operation prior to computing the loss. If already-normalized probabilities (lying in the interval $[0, 1]$) are fed into `CrossEntropyLoss`, the internal softmax function treats them as raw logits. For example, if the network outputs a highly confident probability vector of $[0.99, 0.01]$, the internal softmax maps the larger value (0.99) down to approximately 0.73, while elevating the smaller one (0.01) to 0.27; this mathematical distortion penalizes correct predictions, leading to inefficient learning and poor convergence. In our hybrid architecture, the output of the quantum layer consists of physical probabilities reconstructed from statistical measurements, then, if we wish to employ `CrossEntropyLoss`, we have to pass the raw, unnormalized measurement counts directly to the loss function, rather than converting them into normalized frequencies. Consequently, our framework implements the `NLLLoss` function, which expects log-probabilities as inputs. Generally, classical networks apply a `LogSoftmax` activation to raw logits to obtain these log-probabilities. However, since the outputs of our quantum simulator are already normalized probabilities, we directly compute their element-wise logarithm:

$$C(\hat{y}, y) = \text{NLLLoss}(\log(\hat{y} + \epsilon), y) \quad (2.18)$$

In our implementation, this is achieved through the following PyTorch formulation:

```
loss = loss_func(torch.log(output + 1e-15), target)
```

where a small regularization constant $\epsilon = 10^{-15}$ is added to the probability tensor. This is a numerical precaution to prevent evaluating $\log(0)$, which would otherwise diverge to negative infinity.

Finally, it is worth noting that while the parameter-shift rule has been formally extended to high-dimensional qudit systems, we adopted its simplified qubit-like formulation. Nevertheless, implementing the fully generalized qudit parameter-shift rule represents a natural direction for future work.

2.3 Tests and results

In this section, we present the numerical results obtained by simulating our hybrid classical-quantum neural network with different quantum circuits. The primary objective of these benchmarks is to evaluate the learning capabilities and scaling behavior of a variational quantum layer based on high-dimensional single-qudit systems ($d > 2$) rather than standard binary qubits. To conduct a thorough evaluation, we analyze three key metrics:

- the total training time of the neural network t_{tot} [*seconds*];
- the classification accuracy $\mathcal{A}$ (%) (see Section 1.1.4);
- the training reliability (or success rate).

The training reliability is designed to evaluate the stability of the optimization process. Specifically, it represents the probability that the model’s final classification accuracy exceeds a predefined threshold of 90%.

All the hybrid neural networks evaluated in our tests employ parameterized quantum layers that share the following characteristics:

- Although the coupling graph is already connected and all energy levels are reachable using only $d - 1$ gates, we apply an additional gate to the subspace $\{0, d - 1\}$. While this introduces additional parameters, empirical tests show that this redundant coupling significantly enhances the network’s performance, boosting the average classification accuracy and reliability.
- The variational quantum circuit is positioned as the final output layer. Consequently, the class predictions are determined directly by the quantum measurement probabilities of the qudit states.
- Each circuit is initialized in an equal superposition of the d computational basis states, prepared by applying the generalized Hadamard gate H_d at the input.

Below, we describe the specific neural network configurations evaluated in our tests. To guarantee the complete reproducibility of our results, we associate each model with its corresponding Python file. The exact filename is reported in square brackets directly within the section title, formatted as `[filename]`. All implementation scripts are hosted in our GitHub repository [22].

2.3.1 Quantum layer with [entangled_qubit] circuit

First, we present the benchmarking results obtained by a qubit based hybrid neural network architecture [23] to establish a baseline for our qudit system. Although the original network employed convolutional layers, we replaced them with fully connected (linear) layers to ensure a fair comparison between structurally equivalent models. The quantum component of this architecture utilizes three entangled qubits undergoing three parameterized rotations; by measuring a single qubit, the network performs binary classification on two target labels. The schematic layout of the quantum circuit is shown in Figure 2.2.

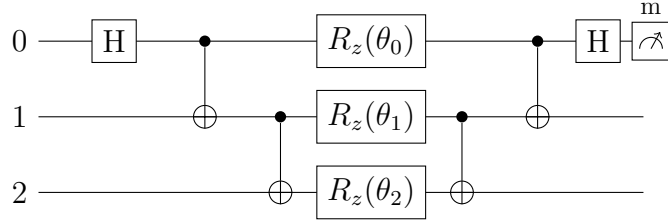


Figure 2.2: Quantum circuit based on entangled qubits

As illustrated in Table 2.1, the network consistently achieves a high classification accuracy exceeding $\mathcal{A} = 99\%$. Given a standard deviation of $\sigma_{\mathcal{A}} = 0.05\%$, all ten independent runs converged over the target threshold, confirming the reproducibility of the optimization process with a 100% success rate.

Table 2.1: Performance results across ten independent experimental runs.

Test n°	1	2	3	4	5	6	7	8	9	10
t_{tot} [s]	121	115	121	119	124	121	114	119	125	112
$\mathcal{A}$ (%)	99.80	99.95	99.90	99.80	99.90	99.90	99.85	99.85	99.95	99.85

2.3.2 Quantum layer with [H_X_ij] circuit

This circuit is designed to operate on a qutrit system ($d = 3$). The circuit architecture is composed of three $X_{ij}(\theta)$ rotation gates, each acting on a specific two-dimensional subspace of the three-level system. Since each rotation is parameterized by a single angle, the quantum layer introduces a total of 3 trainable parameters to the overall network.

As shown in Table 2.2, this model achieves a high level of performance, with the validation accuracy consistently stabilizing around $\mathcal{A} \approx 98.7\%$. Although slightly less accurate than the qubit control circuit, this architecture requires lower computational overhead, with an average execution time of $\bar{t}_{tot} = 102.4$ seconds; furthermore, the expansion of the Hilbert space to $d = 3$ allows the network to classify a third digit label. Its statistical

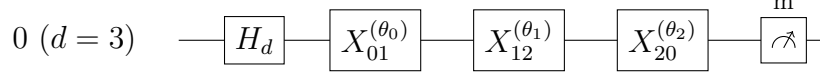


Figure 2.3: Quantum circuit based on single-parameter $X_{ij}(\theta_n)$ rotation gates acting on a three-dimensional qudit

dispersion ($\sigma_{\mathcal{A}} = 0.18\%$) confirms a 100% success rate in convergence across all ten runs of this test, indicating excellent training reproducibility.

Table 2.2: Performance results across ten independent experimental runs.

Test n°	1	2	3	4	5	6	7	8	9	10
t_{tot} [s]	99	101	108	104	98	103	109	107	98	97
$\mathcal{A}$ (%)	98.39	98.83	98.76	98.86	98.49	98.59	98.83	98.83	98.99	98.86

Although alternative configurations such as `H_Y_ij` and `H_Z_ij_H` were also evaluated, they yielded redundant results and offered no additional physical insights beyond the `H_X_ij` architecture.

2.3.3 Quantum layer with [H_X_ij-d=5] circuit

To expand the system’s state-space, this architecture utilizes five single-parameter $X_{ij}(\theta)$ rotation gates acting within a $d = 5$ Hilbert space. The gates are arranged sequentially between adjacent energy levels to ensure the complete connectivity of the qudit states, resulting in exactly 5 trainable parameters. This layout increases the mathematical dimensionality of the quantum layer while maintaining a minimal number of variables.

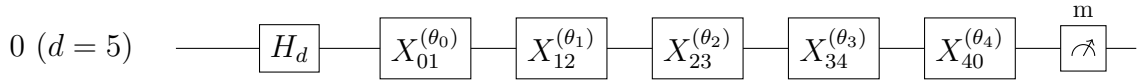


Figure 2.4: Quantum circuit based on single-parameter $X_{ij}(\theta_n)$ rotation gates acting on a five-dimensional qudit

Scaling the qudit dimension to $d = 5$ allows the model to classify five distinct digit classes, but at the expense of both training stability and computational efficiency. As reported in Table 2.3, the classification accuracy undergoes a significant degradation, and the statistical dispersion widens considerably ($\sigma_{\mathcal{A}} = 3.47\%$). Simultaneously, the computational cost increases, with the average training time rising to $\bar{t}_{tot} = 272.2$ seconds, nearly tripling the duration of the $d = 3$ case. Despite these challenges, the network still demonstrates a viable learning capability, as the optimization process converges with an 80% reliability.

Table 2.3: Performance results across ten independent experimental runs.

Test n°	1	2	3	4	5	6	7	8	9	10
t_{tot} [s]	260	255	279	269	268	281	273	275	287	282
$\mathcal{A}$ (%)	93.15	87.06	95.59	86.32	96.67	96.41	95.10	94.44	93.31	91.54

2.3.4 Quantum layer with [H_R_ij] circuit

This qutrit ansatz ($d = 3$) replicates the universal gate set based on generalized Givens rotations proposed in [15]. It applies the rotation gate $R_{ij}(\theta, \phi)$ to each pair of energy levels, connecting the coupling graph. By doubling the variational angles per transition, the model introduces a total of 6 trainable parameters.

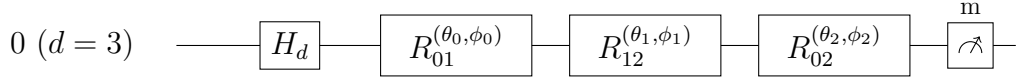


Figure 2.5: Quantum circuit based on generalized Givens rotation gates $R_{ij}(\theta_n, \phi_n)$ acting on a three-dimensional qudit

Introducing the two-parameter R_{ij} rotation gates increases the model’s expressive power, though this does not translate into a tangible performance gain. As shown in Table 2.4 the validation accuracy converges to a stable average ($\mathcal{A} \approx 98.7\%$, $\sigma_{\mathcal{A}} = 0.14\%$) that is virtually identical to that of the single-parameter X_{ij} circuit ($d = 3$). However, the additional parameters incur a higher computational cost, elevating the average training time to $\bar{t}_{tot} = 175.4$ seconds. Notably, this cost remains lower than that of the $d = 5$ single-parameter circuit, and the optimization process preserves a 100% success rate.

Table 2.4: Performance results across ten independent experimental runs.

Test n°	1	2	3	4	5	6	7	8	9	10
t_{tot} [s]	159	189	178	175	190	174	169	163	173	184
$\mathcal{A}$ (%)	98.79	98.76	98.72	98.76	98.76	98.29	98.76	98.76	98.72	98.66

2.3.5 Quantum layer with [H_R_ij_dim=5] circuit

Combining high-dimensional scaling ($d = 5$) with a two-parameter rotation gate, this quantum layer is designed to map inputs to five output classes. Connecting the five energy states via $R_{ij}(\theta, \phi)$ gates yields a total of 10 trainable parameters.

As shown in Table 2.5, the validation accuracy averages $\mathcal{A} \approx 96.4\%$ with a low statistical dispersion ($\sigma_{\mathcal{A}} \approx 1.56\%$). Compared to the single-parameter X_{ij} ($d = 5$) model,

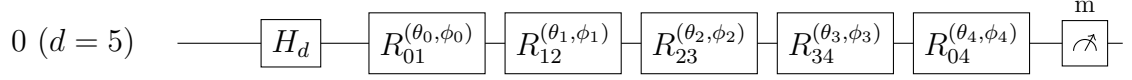


Figure 2.6: Quantum circuit based on generalized Givens rotation gates $R_{ij}(\theta_n, \phi_n)$ acting on a five-dimensional qudit

which suffered from severe performance drops down to 86%, the additional trainable parameters in the R_{ij} gates yield more consistent classification performance. However, this stabilization introduces a heavy computational overhead, with the average training time rising to $\bar{t}_{tot} \approx 530.9$ seconds, nearly doubling the execution time of the X_{ij} ($d = 5$) circuit and making this the most computationally demanding architecture analyzed in this work. Nonetheless, the architecture preserves a 100% convergence rate under this test.

Table 2.5: Performance results across ten independent experimental runs.

Test n°	1	2	3	4	5	6	7	8	9	10
t_{tot} [s]	534	536	559	573	490	513	501	536	540	527
$\mathcal{A}$ (%)	97.12	96.76	96.69	96.41	97.10	97.10	96.90	91.74	97.04	96.76

2.3.6 Quantum layer with [H_XYZ_ij] circuit

This qutrit-based ($d = 3$) circuit emulates the universal single-qubit gate obtained through the Euler decomposition 1.2.1, extending its mathematical structure to the subspaces of a three-level system. Each XYZ_{ij} block is composed of three consecutive elementary rotations acting on the same two-level subspace of the qutrit, with a total of 9 trainable parameters.

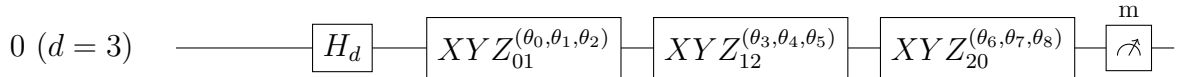


Figure 2.7: Quantum circuit based on a combination of single-parameter rotation gates acting on a three-dimensional qudit. For the sake of clarity, the sequential gates $X_{ij}^{(\theta_l)}$, $Y_{ij}^{(\theta_m)}$, and $Z_{ij}^{(\theta_n)}$ acting on the same subspace have been condensed into a single $XYZ_{ij}^{(\theta_l, \theta_m, \theta_n)}$ block

The condensed XYZ configuration represents the most parameter-dense ansatz for the $d = 3$ system, enabling the network to occasionally reach a peak validation accuracy of 99.09%. As illustrated in Table 2.7, the average accuracy is stable with low dispersion ($\sigma_{\mathcal{A}} = 0.16\%$), yielding a 100% training reliability under this test. However, this high stability comes at a severe computational cost: the average training time surges to

$\bar{t}_{tot} = 319.4$ seconds, making it the most computationally expensive model among those tested for $d = 3$.

Table 2.6: Performance results across ten independent experimental runs.

Test n°	1	2	3	4	5	6	7	8	9	10
t_{tot} [s]	336	332	328	324	322	329	306	306	305	306
$\mathcal{A}$ (%)	98.79	98.72	98.76	98.62	99.09	98.93	98.62	98.46	98.72	98.72

Hyperparameters and Network Characteristics

All the networks implemented shares the same simulation parameters summarized in Table 2.7. The parameter Global Shots is defined as the number of independent circuit executions required to reconstruct the qudit state probabilities, while the other parameters have been addressed in previous sections. The learning rate and number of epochs were selected empirically to ensure stable training convergence.

Table 2.7: Hyperparameters and simulation configurations of all implemented networks.

Parameter	Value
Global Shots	1000
Shift (s)	$\pi/2$
Learning Rate (η)	0.001
Epochs	15
Loss Function	<code>nn.NLLLoss()</code>
Optimizer	Adam

Chapter 3

Discussion and Outlook

In summary, this thesis evaluated the learning characteristics of hybrid quantum-classical neural networks using a qudit-based quantum circuit as a network layer to classify MNIST digits. To evaluate the learning capabilities of multi-level quantum layers, we benchmarked five distinct qudit-based variational circuit configurations, comparing them against a binary qubit control configuration:

- `qubit` ($d = 2$): The baseline qubit model featuring three parameters;
- `H_X_ij` ($d = 3$): A qutrit model featuring three single-parameter subspace rotations;
- `H_X_ij_d=5` ($d = 5$): A five-dimensional qudit model utilizing five single-parameter subspace rotations;
- `H_R_ij` ($d = 3$): A qutrit model based on three two-parameter generalized Givens rotations;
- `H_R_ij_dim=5` ($d = 5$): A five-dimensional qudit model employing five two-parameter generalized Givens rotations;
- `H_XYZ_ij` ($d = 3$): A qutrit model featuring three three-parameter subspace rotations emulating the Euler decomposition.

3.1 Discussion of Experimental Results

The empirical distribution of the data in Figure 3.1 highlights several key physical and computational aspects of our architectures. The single-parameter `H_X_ij` circuit achieves an average validation accuracy of 98.7% in about 102 seconds, standing out as the most computationally efficient qudit model. Interestingly, as we increase the number of parameters to 6 parameters in `H_R_ij` and 9 parameters in `H_XYZ_ij`, the average accuracy

remains virtually unchanged. Instead, the models scale vertically along the time axis, with the average training times rising to 175.4 seconds and 319.4 seconds, respectively. Therefore, adding parameterized rotations within a low-dimensional Hilbert space does not improve expressiveness, but only introduces computational cost. In contrast, transi-

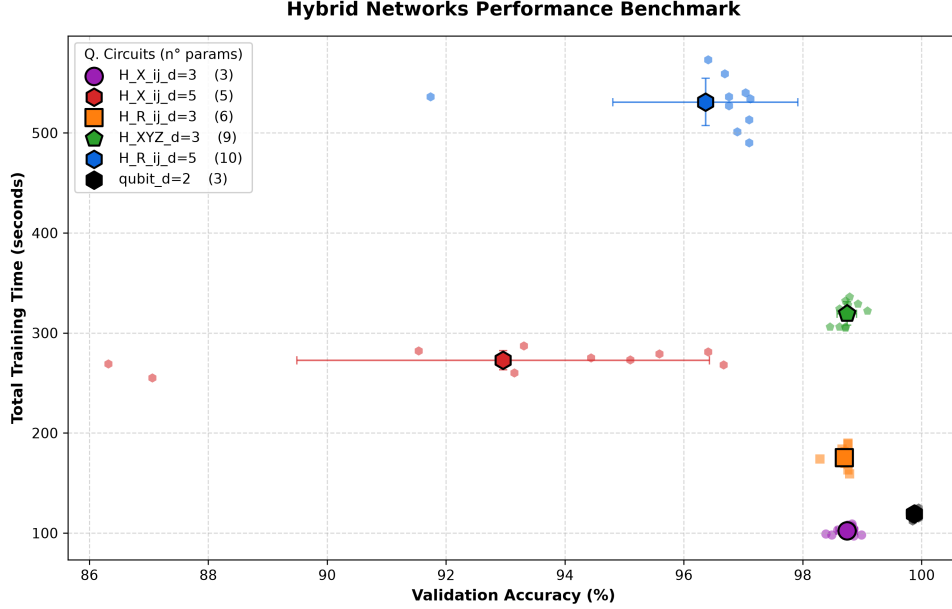


Figure 3.1: The plot illustrates the trade-off between accuracy and total training time across 10 independent runs for each circuit. Semi-transparent markers represent individual experimental runs, while large solid markers denote the calculated averages. Horizontal and vertical error bars indicate the standard deviation for accuracy and training time, respectively.

tioning to $d = 5$ reveals a completely different situation. The single-parameter $H_X_ij_d=5$ model is unstable, characterized by a diminished average accuracy of 92.96% and a broad horizontal error bar representing a significant statistical dispersion ($\sigma_A \approx 3.47\%$). However, by doubling the parameter density with the generalized Givens rotation ansatz in $H_R_ij_dim=5$, the system is better stabilized. This richer parametrization restores the model’s training reliability to 100% and narrows the statistical dispersion, bringing the average accuracy to 96.36% ($\sigma_A \approx 1.56\%$). This indicates that while simple single-parameter rotations are sufficient to navigate smaller Hilbert spaces, richer parameterized ansatzes are necessary to optimize higher-dimensional qudit systems. This stabilization, as visually shown in Figure 3.1, requires a major computational sacrifice, positioning $H_R_ij_dim=5$ as the most expensive architecture tested, with a training time averaging 530.9 seconds.

Finally, the binary $qubit_d=2$ baseline (black hexagon) consistently converges to an accuracy of 99.9% in approximately 120 seconds. Although the qudit models do not sur-

pass this baseline in absolute accuracy or execution speed, they still demonstrate solid performance. More importantly, while the qubit is restricted to binary classification, the qutrit ($d = 3$) and qudit ($d = 5$) systems expand this capability to three and five distinct digit classes using a single physical carrier.

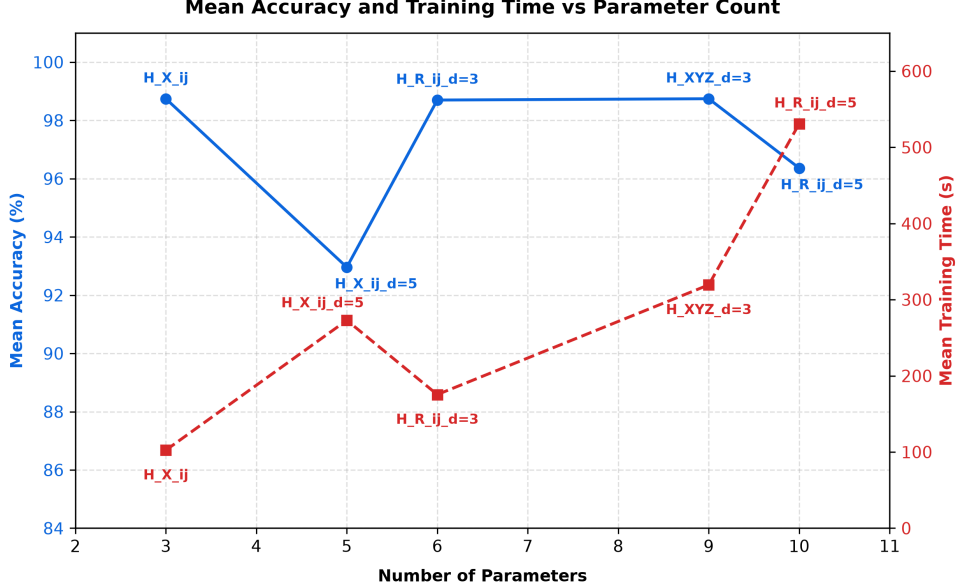


Figure 3.2: Mean accuracy and mean training time as functions of the number of trainable parameters in the variational quantum circuit. Mean accuracy and training time values are calculated across 10 independent training runs for each circuit configuration.

Looking at Figure 3.2, we observe that the number of trainable parameters of the quantum layer affects classification accuracy and computational execution time in different ways. First, there is no direct correlation between the number of variational parameters and the final classification accuracy. The performance is fundamentally governed by the specific dimensionality d and the structure of the circuit rather than the raw parameter count. Conversely, a clear correlation emerges between the total training time and the parameter count. This behavior is directly attributable to the computational mechanics of the parameter-shift rule. Evaluating the exact quantum gradient of a loss function with respect to a single variational parameter θ_k requires executing two distinct forward passes of the quantum circuit at shifted coordinates ($\theta_k \pm \pi/2$). Consequently, with the number of shots set to $N_{\text{shots}} = 1000$ to compute the statistical measurement over the circuit, optimizing each additional parameter demands exactly 2000 extra physical executions of the quantum circuit per epoch. As shown in Figure 3.2, this leads to an increase in training duration, making highly parameterized qudit models like H_R_ij_dim=5 (10 parameters)

expensive to train.

In Figure 3.2, the qutrit circuit $H_{R_{ij}}$ ($d = 3$, 6 parameters) performs better than the qudit circuit $H_{X_{ij}, d=5}$ ($d = 5$, 5 parameters), despite their comparable parameter counts. We hypothesize that this discrepancy stems from how quantum information and population are transferred across the energy levels. In the R_{ij} ansatz ($d = 3$), the gates establish a fully connected coupling structure where every energy level is directly linked to every other level, enabling direct transfer of population and phase information across the entire Hilbert space. Conversely, the single-parameter X_{ij} circuit ($d = 5$) connects five levels using only five gates, resulting in a closed ring-like coupling structure. In this configuration, the transfer of quantum information between non-adjacent energy levels is indirect and requires multiple sequential gate operations. This structural constraint provides a plausible explanation for the severe training instability and lower accuracy observed for $H_{X_{ij}, d=5}$.

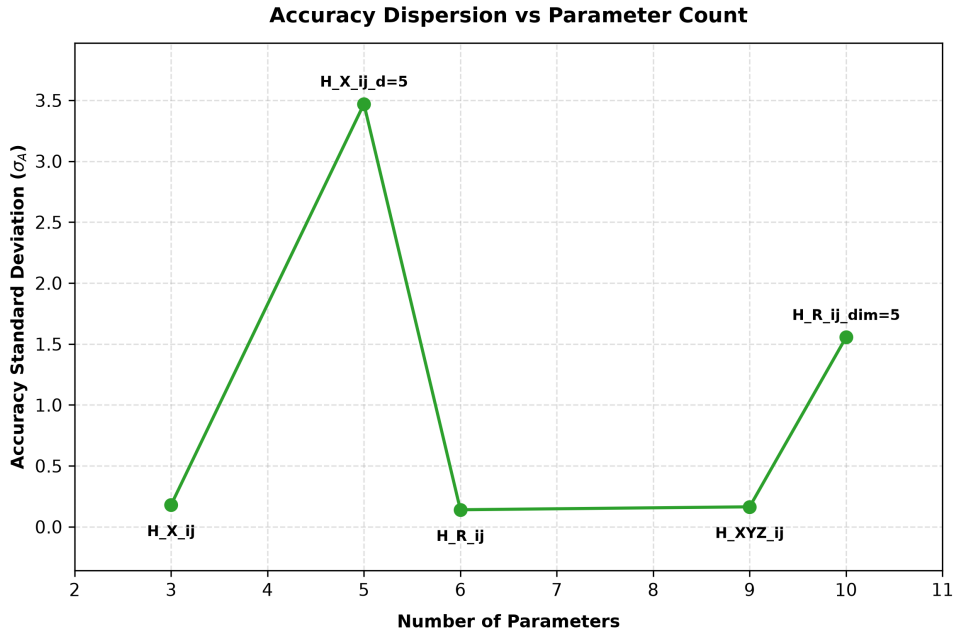


Figure 3.3: Statistical dispersion of the validation accuracy, expressed as the standard deviation (σ_A) calculated across 10 independent training runs, as a function of the number of parameters in the variational quantum circuit.

This hypothesis is further supported by the dispersion analysis in Figure 3.3, where the standard deviation of accuracy ($\sigma_{\mathcal{A}}$) serves as a direct metric of optimization stability. Specifically, all $d = 3$ configurations, which share a fully connected coupling graph, exhibit a low and constant dispersion ($\sigma_{\mathcal{A}} \approx 0.16\%$). Conversely, the ring-like `H.X.ij.d=5` circuit displays significant instability ($\sigma_{\mathcal{A}} \approx 3.47\%$), directly reflecting the bottleneck of its constrained connectivity. This instability is partially mitigated in `H.R.ij.dim=5` ($\sigma_{\mathcal{A}} \approx 1.56\%$), where doubling the parameter density per transition increases gate expressiveness. Overall, these trends reinforce the hypothesis that limited level connectivity may induce training instability, which could potentially be compensated for by increasing gate expressiveness.

3.1.1 The role of Generalized Hadamard Gate

A critical design choice across all implemented circuits was the initialization of the qudit state in a balanced coherent superposition using a generalized Hadamard gate (H_d). Theoretically, preparing this superposition is required to prevent the relative quantum phases (ϕ) from vanishing upon projective measurement. To critically evaluate this assumption, auxiliary benchmarks were executed by training the circuits directly from the computational ground state $|0\rangle$, omitting the initial H_d gate. It was observed that all five proposed architectures converged to stable solutions even without Hadamard initialization. The practical performance gain provided by the H_d gate was modest, yielding an increase of barely 1.0% in final accuracy, and inducing no observable variations in either training time or optimization reliability.

While keeping the H_d operator is beneficial in classical software simulations to maximize the model’s expressive power, its physical implementation would represent an additional cost. Our empirical findings suggest that in a resource-constrained physical implementation, the generalized Hadamard gate could be safely bypassed, at the expense of only a negligible penalty in final classification performance.

3.1.2 Future Work

These findings leave open several interesting questions regarding both the optimization and the physical realizability of qudit-based models. Future work could develop along a few main lines:

- **Validation of the coupling graph hypothesis:** Future work should evaluate alternative ansatzes with higher connectivity, to test whether fully connected level-coupling graphs, compared to ring graphs, enable more efficient quantum information and population transfer across energy levels.
- **Tailored qudit circuit design:** While the variational gates in this study were primarily chosen for their practical convenience, future research should investigate

and design quantum circuits specifically optimized for high-dimensional machine learning tasks.

- **Integration with convolutional layers:** To scale the network’s processing capability, the single-qudit variational layer could be integrated into a hybrid convolutional architecture. This would involve placing classical convolutional layers, accompanied by standard deep learning techniques such as max pooling and dropout, before the quantum layer to improve feature extraction and mitigate overfitting.
- **Simulation and software optimization:** Since the recorded training times are dependent on local workstation performance, executing the simulations on high-performance computing infrastructures, could significantly reduce computational bottlenecks.
- **Noise modeling and robustness analysis:** To evaluate the model’s resilience in realistic, noisy environments, future simulations should introduce physical noise models—such as Gaussian amplitude damping or dephasing channels—utilizing Cirq’s built-in noise simulation modules.
- **Execution on physical quantum hardware:** Future research could focus on analyzing the physical implementability of qudit circuits, designing viable architectures, and subsequently validating them on real quantum processors.

3.2 Conclusions

In this thesis, we investigated the implementation of qudits in Quantum Machine Learning by designing a hybrid quantum-classical neural network for MNIST digit classification. Such a model is obtained by embedding a variational quantum circuit as a functional layer within a classical neural network architecture. Rather than relying on standard binary qubits ($d = 2$), the quantum layer was constructed using d -level qudits ($d > 2$). This choice enables vertical scaling by expanding the local Hilbert space $\mathcal{H}_d$ on a single physical carrier, thereby increasing information density. Indeed, high-dimensional qudits directly expand the multi-class classification capacity of a single physical unit: while a single qubit is fundamentally restricted to binary classification, a single d -dimensional qudit can directly map output measurement probabilities to d distinct digit classes.

Since standard qubit gates cannot be directly extended to d -level systems, defining qudit operators demanded dedicated theoretical formalization alongside custom software development. Mathematically, variational operations were formulated as localized rotations acting on two-dimensional target subspaces $\{|i\rangle, |j\rangle\}$, encompassing single-parameter rotations ($X_{ij}(\theta), Y_{ij}(\theta), Z_{ij}(\theta)$) and two-parameter generalized Givens rotations ($R_{ij}(\theta, \phi)$). Furthermore, the qudit was prepared in a fully balanced initial superposition state via a generalized Hadamard gate H_d . To ensure gradient-based trainability, circuit ansatzes were designed with a connected level-coupling graph, allowing every energy level to exchange population with the others across all basis states. This was achieved by chaining localized two-level gates between adjacent energy levels.

The experimental results confirmed that a single qudit can successfully execute multi-class image classification. For qutrit systems ($d = 3$), all evaluated circuits achieved high accuracy ($\approx 98.7\%$) and remarkable stability ($\sigma_{\mathcal{A}} \approx 0.16\%$), with 100% training reliability. In contrast, for five-level qudits ($d = 5$), the single-parameter X_{ij} architecture exhibited optimization instability ($\sigma_{\mathcal{A}} \approx 3.47\%$, mean accuracy 93.0%). Doubling the parameter density in the two-parameter Givens rotation ansatz (R_{ij}) increased circuit expressiveness, effectively mitigating this instability ($\sigma_{\mathcal{A}} \approx 1.56\%$, mean accuracy 96.4%). However, this gain in stability incurred a steep increase in training time due to the computational cost of evaluating gradients via the parameter-shift rule.

For $d = 3$, cyclic nearest-neighbor operations naturally form a fully connected graph, enabling direct transitions between all states. In contrast, for $d = 5$, the same scheme yields a ring graph, where population transfer between non-adjacent levels must proceed through intermediate states, which may hinder optimization unless additional couplings are introduced. Thus, we hypothesize that the performance discrepancy between systems of different dimensions stems from the connectivity of the level-coupling graphs. Ultimately, these findings demonstrate that high-dimensional single-qudit architectures offer a viable path for compact quantum machine learning; however, they do not immediately outperform standard qubit-based models.

Bibliography

- [1] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. <http://www.deeplearningbook.org>.
- [2] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer Series in Operations Research and Financial Engineering. Springer, New York, 2 edition, 2006.
- [3] Suraj Subramanian, Seth Juarez, Cassie Breviu, Dmitry Soshnikov, and Ari Bornstein. *PyTorch, Learn the Basics*. <https://docs.pytorch.org/tutorials/beginner/basics/intro.html>, 2021. Last Updated: Jan 20, 2026. Accessed: Sep 9, 2026.
- [4] *The N-dimensional array (ndarray)*. <https://numpy.org/doc/stable/reference/arrays.ndarray.html>. Accessed: Sep 9, 2026.
- [5] *Softmax function*. <https://docs.pytorch.org/docs/2.14/generated/torch.nn.Softmax.html>. Accessed: Sep 9, 2026.
- [6] *What is quantum machine learning?* <https://pennylane.ai/whatisqml>. Accessed: Sep 9, 2026.
- [7] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 2010.
- [8] Yuchen Wang, Zixuan Hu, Barry C. Sanders, and Sabre Kais. Qudits and high-dimensional quantum computing. *Frontiers in Physics*, Volume 8 - 2020, 2020.
- [9] *Google Cirq*. <https://quantumai.google/cirq1>. Accessed: Sep 9, 2026.
- [10] Santanu Pattanayak. *Quantum Machine Learning with Python - Using Cirq from Google Research and IBM Qiskit*. 2021.
- [11] *Parameter Shift Rule*. https://pennylane.ai/glossary/parameter_shift#a-more-technical-explanation-. Accessed: Sep 9, 2026.

- [12] Rodolfo Carobene. Beyond qubits: A comprehensive survey of the current state of qudit research. 2026.
- [13] Ashok Muthukrishnan and C. R. Stroud. Multivalued logic gates for quantum computation. *Physical Review A*, 62(5), October 2000.
- [14] Gavin Brennen, Dianne O’Leary, and Stephen Bullock. Criteria for exact qudit universality. *Physical Review A*, 71(5), May 2005.
- [15] Sebastián Roca-Jerat, Juan Román-Roche, and David Zueco. Qudit machine learning. *Machine Learning: Science and Technology*, 5(1):015057, mar 2024.
- [16] Francesco Pudda, Mario Chizzini, and Luca Crippa. Generalised quantum gates for qudits and their application in quantum fourier transform, 2026.
- [17] *Qudits with Cirq*. <https://quantumai.google/cirq/build/qudits>. Accessed: Sep 9, 2026.
- [18] *cirq.Param.Resolver*. <https://quantumai.google/reference/python/cirq/ParamResolver>. Accessed: Sep 9, 2026.
- [19] *Cross Entropy Loss function*. <https://docs.pytorch.org/docs/2.14/generated/torch.nn.CrossEntropyLoss.html>. Accessed: Sep 9, 2026.
- [20] *Negative Log Likelihood function*. <https://docs.pytorch.org/docs/2.14/generated/torch.nn.NLLLoss.html>. Accessed: Sep 9, 2026.
- [21] *LogSoftmax function*. <https://docs.pytorch.org/docs/2.14/generated/torch.nn.LogSoftmax.html>. Accessed: Sep 9, 2026.
- [22] Lorenzo Sabadini. *GitHub repository SQuBa-Class*. <https://github.com/qismib/SQuBa-Class/tree/main>.
- [23] Gaia Stella Bolognini. *GitHub repository TraNQI*. <https://github.com/qismib/TraNQI>.